

NIT2112

Object Oriented Programming

Session 01

Introduction to Object-Oriented Programming & AI Co-Pilot

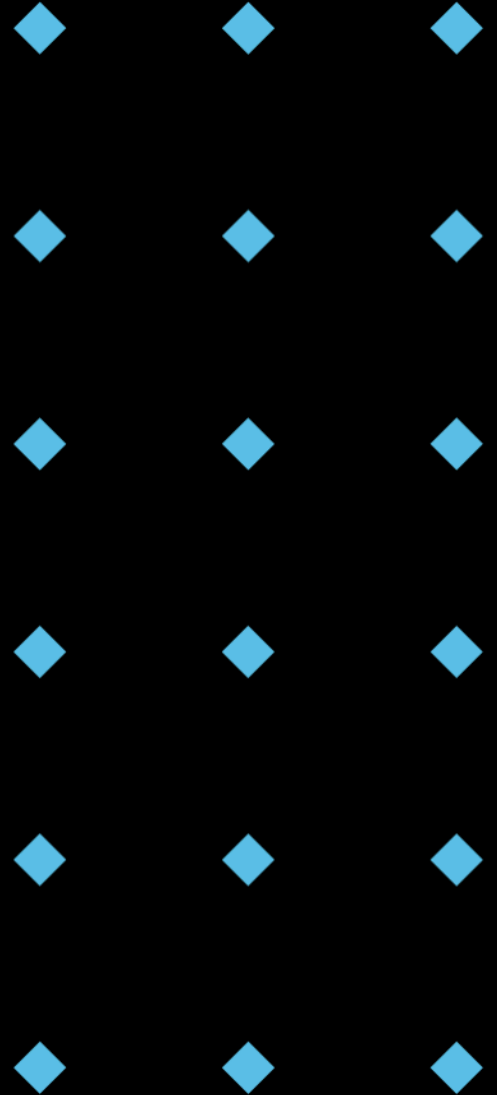
Acknowledgement of Country



Victoria University acknowledges, recognises and respects the Ancestors, Elders and families of the Bunurong/Boonwurrung, Wadawurrung and Wurundjeri/Woiwurrung of the Kulin who are the traditional owners of University land in Victoria, the Gadigal and Guring-gai of the Eora Nation who are the traditional owners of University land in Sydney, and the Yugara/YUgarapul people and Turrbal people living in Meanjin (Brisbane).

Session Outline

- ◆ Python Basics Recap
- ◆ Introduction to OOP Concepts & Benefits
- ◆ Class & Object Fundamentals (Definition, Attributes, Methods, Constructor)



Why Python for OOP?

- ◆ **Clean and Readable Syntax:** Python's emphasis on readability makes complex OOP concepts easier to grasp and implement.
- ◆ **Dynamic Typing:** Flexible, though requires careful management (OOP helps!).
- ◆ **Extensive Standard Library:** Many built-in modules simplify development.
- ◆ **Large Community & Ecosystem:** Abundant resources, libraries (especially for AI!), and support.
- ◆ **Naturally Object-Oriented:** Everything in Python is an object (even simple data types), making OOP feel integrated.

Top programming languages on GitHub

RANKED BY COUNT OF DISTINCT USERS CONTRIBUTING TO PROJECTS OF EACH LANGUAGE.

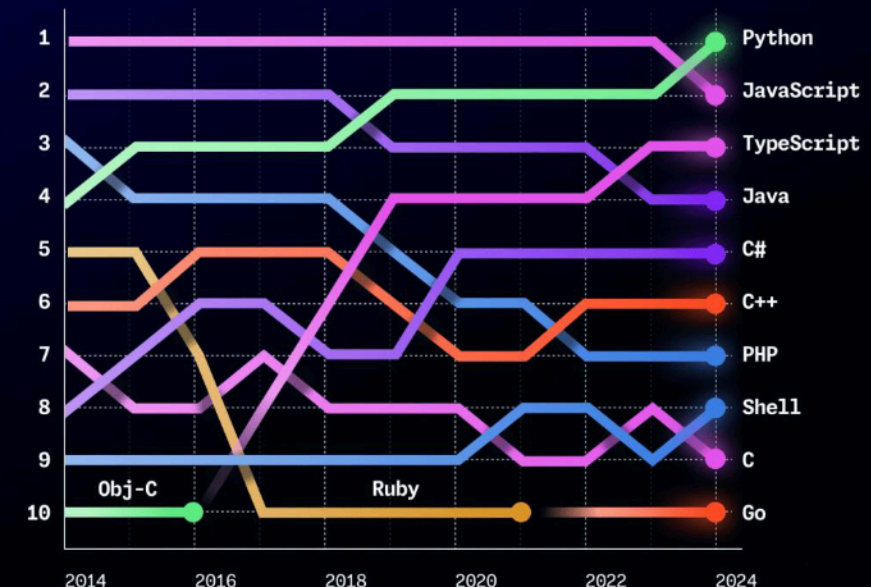
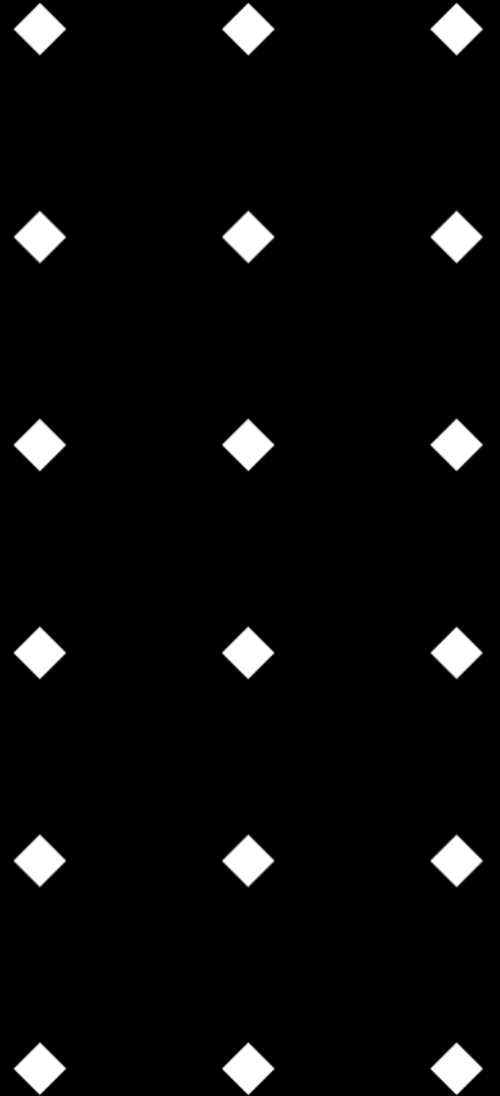


Image from GitHub Blog, October 29, 2024, <https://github.blog/news-insights/octoverse/octoverse-2024/>

Important Note on AI Copilot Usage

- ◆ Foundational Learning (Sessions 1-6):
 - ◆ For these initial sessions covering core OOP concepts (Encapsulation, Inheritance, Polymorphism, etc.), please complete all coding examples and exercises **MANUALLY**.
 - ◆ **DO NOT** use AI Copilot tools for assistance during Sessions 1-6.
- ◆ AI Copilot Integration (Sessions 7-10):
 - ◆ We will specifically cover how to effectively use AI Copilots (like GitHub Copilot, ChatGPT, etc.) to assist with OOP development later in this unit.
 - ◆ This dedicated focus will begin in Session 7.
- ◆ Why?
 - ◆ Mastering the fundamental concepts and syntax through **your own effort** is crucial for **deep understanding**.
 - ◆ Building this foundation first will allow you to use AI tools more **effectively** and **critically** later.

Python Basics



Basic Syntax: Indentation Matters!

- ◆ **Code Blocks:** Python uses **indentation** (spaces or tabs) to define code blocks (like loops, function bodies, class definitions). No curly braces {}!
- ◆ **Consistency is Key:** Use 4 spaces per indentation level (standard practice). Mixing tabs and spaces will cause errors.

```
1  # This is a comment
2  variable_name = 10 # Assignment
3  if variable_name > 5:
4      print("Greater than 5") # Indented block
5      print("Inside the if block")
6  else:
7      print("Not greater than 5") # Another indented block
8  print("Outside the if/else block") # Back to previous level
```

Basic Syntax: Variables and Assignment

- ◆ Variables: Names that refer to values stored in memory.
- ◆ Assignment: Use the equals sign = to assign a value to a variable.
- ◆ Naming Conventions:
 - ◆ Use descriptive names (e.g., *student_name* instead of *sn*).
 - ◆ Start with a letter or underscore _.
 - ◆ Can contain letters, numbers, underscores.
 - ◆ Case-sensitive (*age* is different from *Age*).
 - ◆ Use *snake_case* (lowercase with underscores) for variables and functions (standard Python convention).

```
1 student_count = 25
2 course_name = "Introduction to Python"
3 average_score = 85.5
4 is_active = True
```

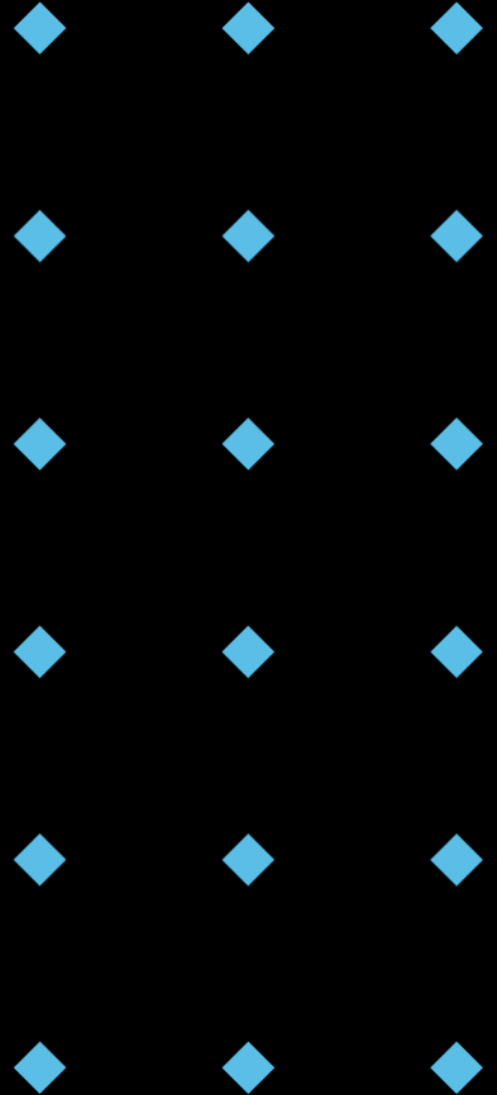

Data Types: Primitive Types

- ◆ **Integers (int):** Whole numbers (e.g., 10, -5, 0).
- ◆ **Floating-Point Numbers (float):** Numbers with a decimal point (e.g., 3.14, -0.5, 2.0).
- ◆ **Booleans (bool):** Truth values, either *True* or *False* (capitalized!). Used heavily in control flow.
- ◆ **Strings (str):** Sequences of characters, enclosed in single (') or double (") quotes.

```
1 age = 30          # int
2 price = 99.99     # float
3 is_enrolled = True # bool
4 student_name = "Alice" # str
5 course_code = 'CS101' # str
```

Data Types: Common Collections

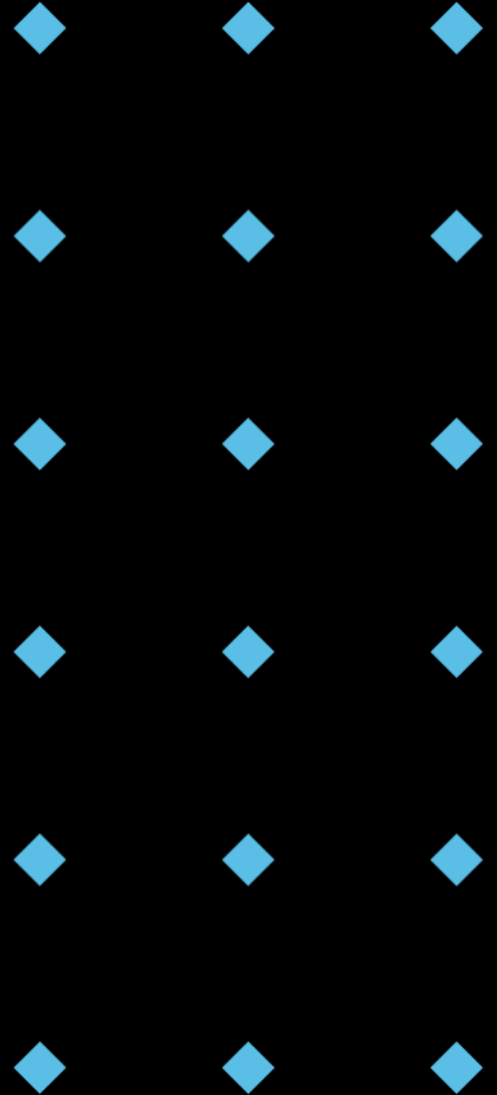
- ◆ **Lists (list):** Ordered, mutable (changeable) sequences. Defined with [].
 - ◆ `my_list = [1, "hello", 3.14, True]`
- ◆ **Tuples (tuple):** Ordered, immutable (unchangeable) sequences. Defined with ().
 - ◆ `my_tuple = (1, "hello", 3.14, True)`
- ◆ **Dictionaries (dict):** Unordered (in older Python versions, ordered in 3.7+) collections of key-value pairs. Defined with {}. Keys must be unique and immutable.
 - ◆ `student = {"name": "Bob", "id": 123, "major": "CS"}`
- ◆ **Sets (set):** Unordered collections of unique elements. Defined with {} but without key-value pairs, or using `set()`.
 - ◆ `unique_numbers = {1, 2, 3, 2, 1} # Results in {1, 2, 3}`



Interactive Question

- Question: What will be the data types of the following variables?

```
1 a = 5
2 b = "5"
3 c = 5.0
4 d = [5]
5 e = (5,) # Note the comma!
6 f = {"age": 5}
7 g = {5}
8 h = (a == c)
```



Control Flow: if, elif, else

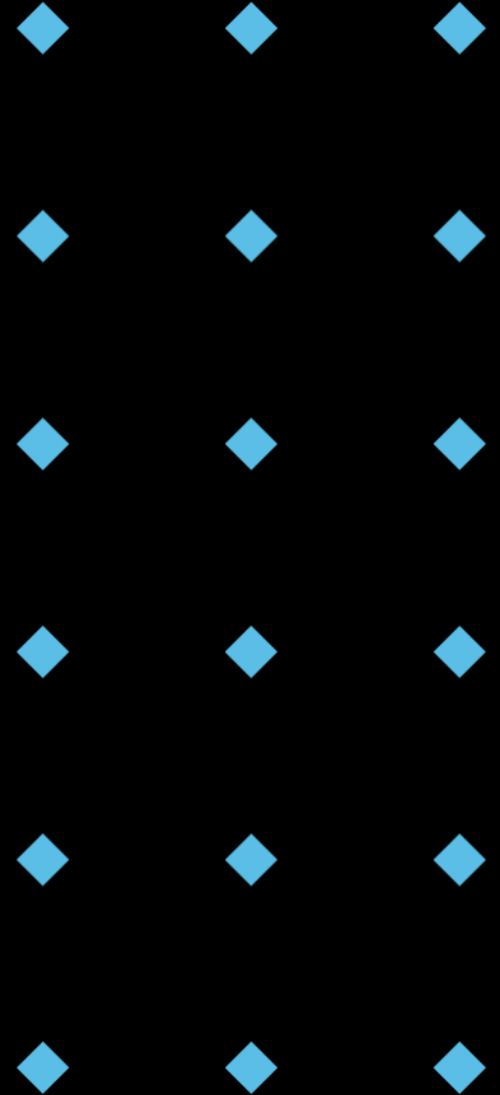
- ◆ If-else: Execute code conditionally based on whether expressions are True or False.

```
1  score = 75
2
3  if score >= 80:
4      print("Grade: HD")
5  elif score >= 70: # Optional: "else if"
6      print("Grade: D")
7  elif score >= 60:
8      print("Grade: C")
9  elif score >= 50:
10     print("Grade: P")
11 else: # Optional: Catches anything not caught above
12     print("Grade: F")
13
14 print("Finished grading.") # Executes regardless
```

Control Flow: for Loops

- ◆ Purpose: Iterate over a sequence (like a list, tuple, string, or range).

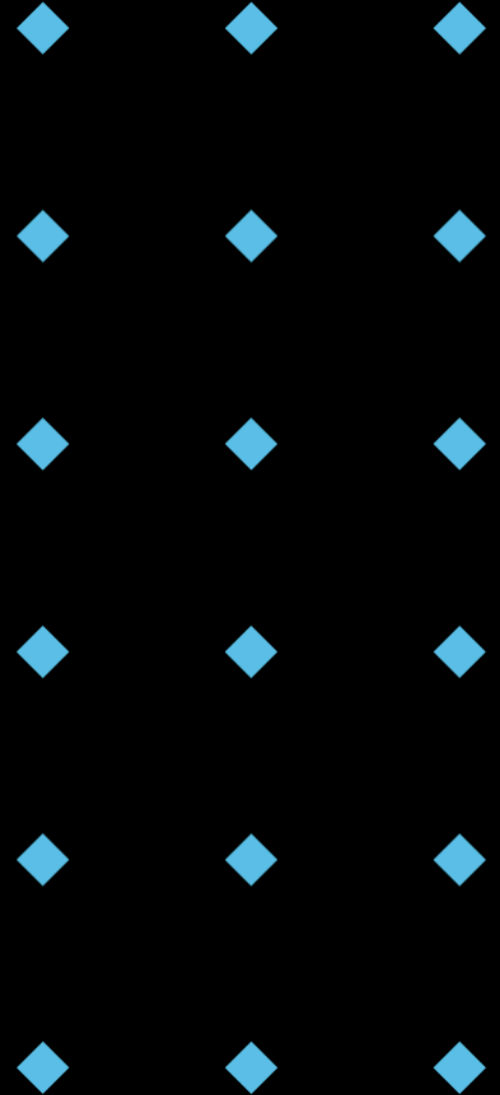
```
1  # Iterate over a sequence
2  # (like a list, tuple, string, or range).
3  fruits = ["apple", "banana", "cherry"]
4  for fruit in fruits:
5      print(fruit.capitalize())
6
7  # Using range
8  # Prints numbers 0, 1, 2, 3, 4
9  for i in range(5):
10     print(f"Number: {i}")
11
12 # Prints numbers 2, 3, 4
13 for j in range(2, 5):
14     print(f"Number between 2 and 5: {j}")
```



Control Flow: while Loops

- ◆ Purpose: Repeat a block of code as long as a condition remains True.

```
1 count = 0
2 while count < 3:
3     print(f"Count is: {count}")
4     count = count + 1 # Crucial: Update the condition variable!
5
6 print("While loop finished.")
```



Functions

- ◆ Purpose: Group reusable blocks of code, making programs organized, modular, and easier to maintain.

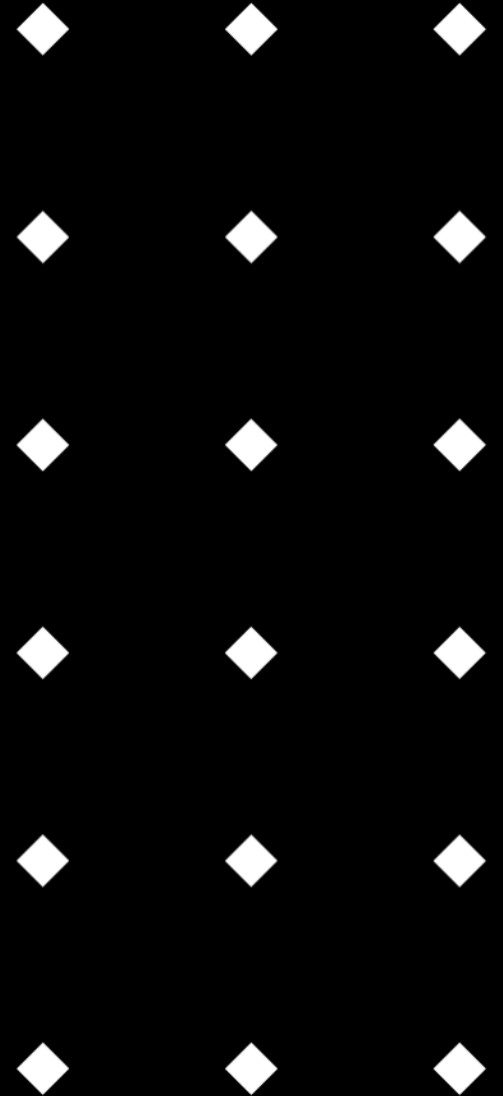
```
1  def greet(name): # Define function 'greet' with parameter 'name'
2      """This function greets the person passed in as a parameter.""" # Docstring
3      print(f"Hello, {name}!")
4
5  def add_numbers(x, y): # Function with two parameters
6      """This function returns the sum of two numbers."""
7      result = x + y
8      return result # Return a value
9
10 # Calling the functions
11 greet("Class")
12 sum_result = add_numbers(5, 3)
13 print(f"The sum is: {sum_result}")
```

Modules

- ◆ Purpose: Organize related functions, classes, and variables into separate files (.py files). Promotes code reusability and modularity.
- ◆ Using Modules: Use the *import* keyword.

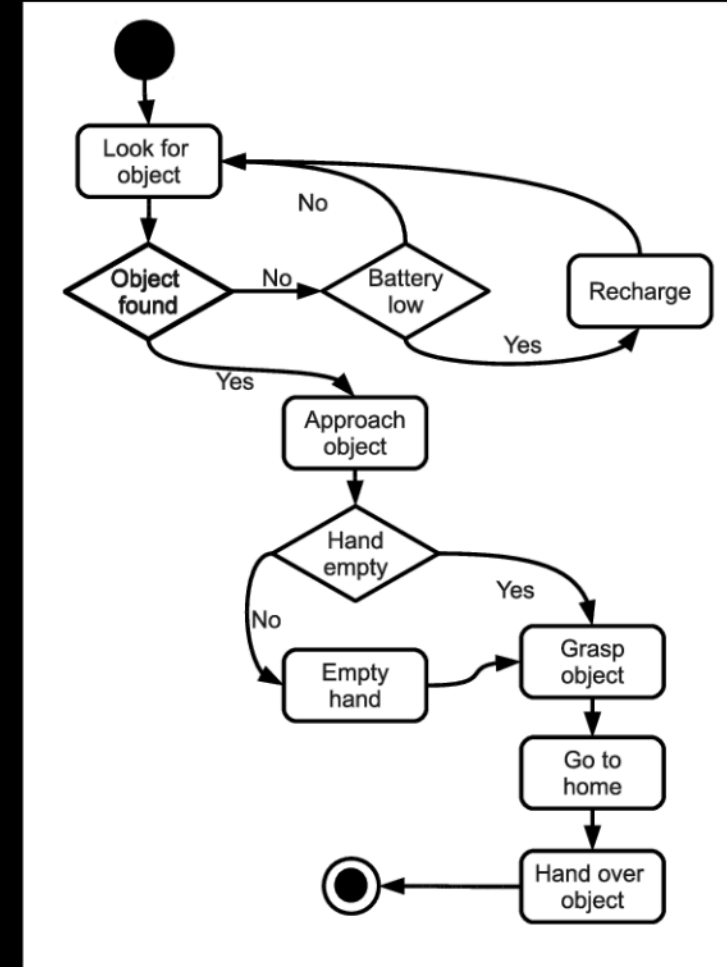
```
1  # Example 1: Import the whole module
2  import math
3
4  print(math.sqrt(16)) # Access functions using module_name.function_name
5  print(math.pi)      # Access constants
6
7  # Example 2: Import specific items from a module
8  from math import sqrt, pi
9
10 print(sqrt(25)) # Access imported items directly
11 print(pi)
12
13 # Example 3: Import with an alias (shorter name)
14 import math as m
15
16 print(m.sqrt(36))
```


Introduction to OOP



Procedural Programming

- ◆ Focus: **Procedures** (routines, subroutines, functions).
- ◆ Structure: A series of computational **steps** to be carried out. Programs are divided into functions. Data tends to be **separate** from the functions that operate on it.
- ◆ Example Languages (Historically): C, Fortran, Pascal. (Python can be used procedurally).
- ◆ Analogy: Following a recipe **step-by-step**. The ingredients (data) are listed separately from the instructions (functions).



Taipalus, Tapio. "Actionpool: A novel dynamic task scheduling method for service robots." (2010).

Object-Oriented Programming (OOP)

- ◆ Focus: "Objects" that bundle **data** and **behaviour** together.
- ◆ Structure: Programs are designed around interacting objects. Data (**attributes**) and the functions that operate on that data (**methods**) are combined into single units (**objects**).
- ◆ Example Languages: Java, C++, C#, Python, Go.
- ◆ Analogy: Building a zoo with dogs. Each dog (object) knows its own **properties** (name, colour, whether thirsty - data) and what it can **do** (drink water or talk to other dogs - behaviour).

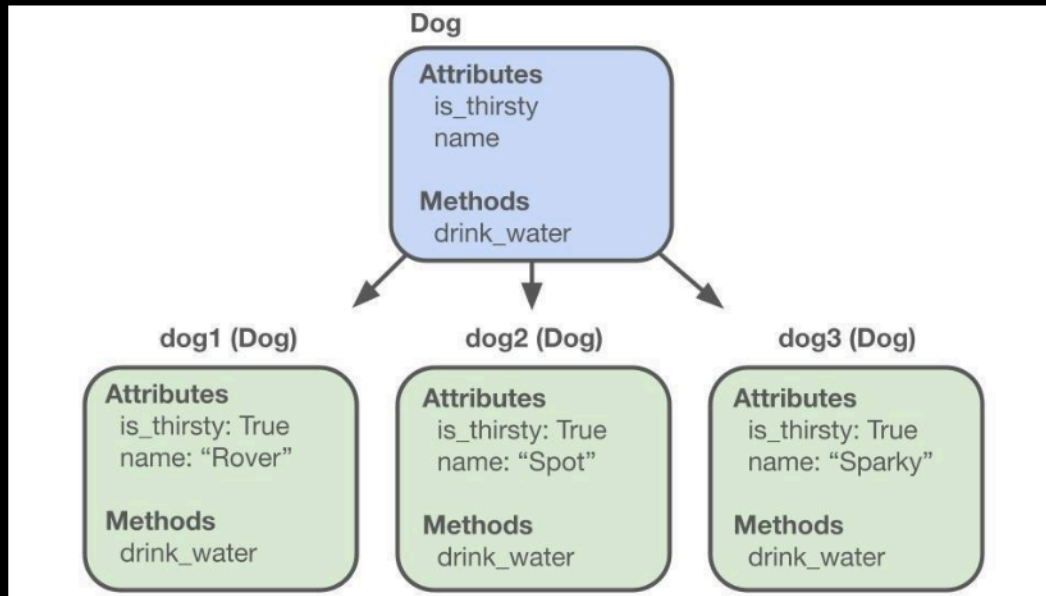
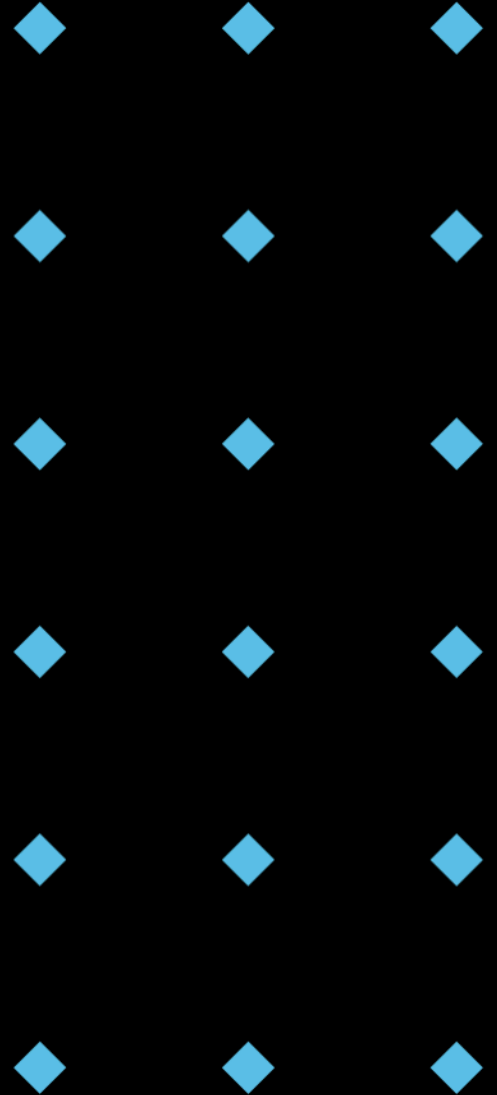


Image from Christina Ellis, July 30, 2021, <https://crunchingthedata.com/cs01-object-oriented-python/>

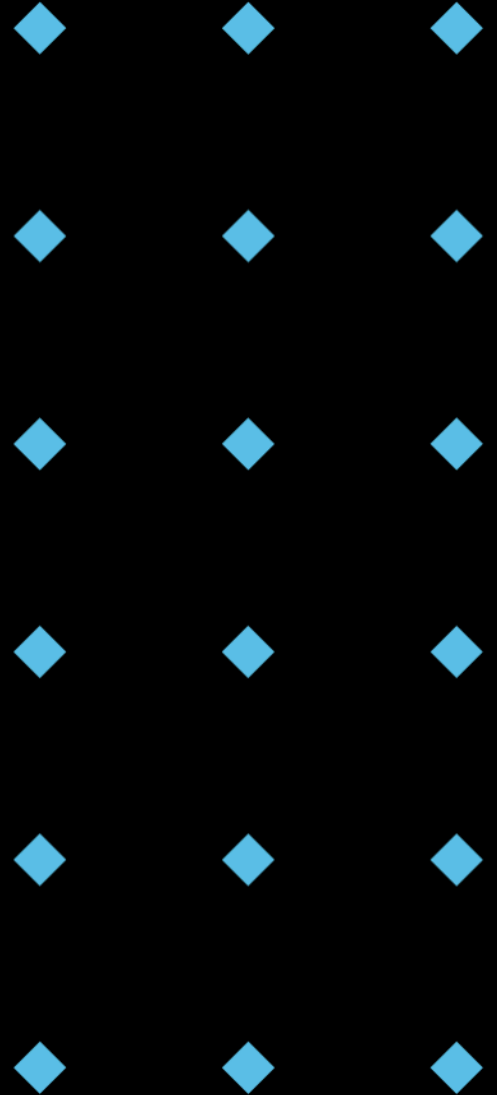
Shifting Perspective: From Actions to Actors

- ◆ **Procedural:** What **actions** need to happen? (e.g., *calculate_pay()*, *update_record()*, *print_report()*)
- Data is passed to functions.
- ◆ **OOP:** What are the '**things**' (objects) involved, and what can they **do**? (e.g., an Employee object that can *calculate_pay()*, an *Account* object that can *deposit()* or *withdraw()*). - Objects perform actions using their own data.



Interactive Discussion

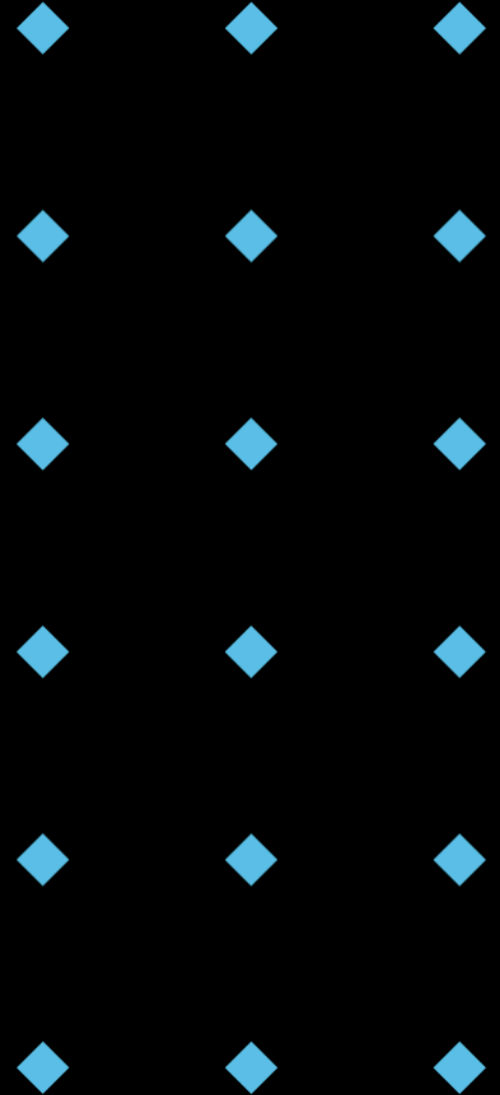
- ◆ Question: Think about a real-world system like a University Enrolment System. What could be some of the key "objects" involved? What data might they hold (attributes)? What actions might they perform (methods)?
- ◆ Examples to prompt: Student, Course, Instructor, Enrolment...



Core Principles of OOP (Introduction)

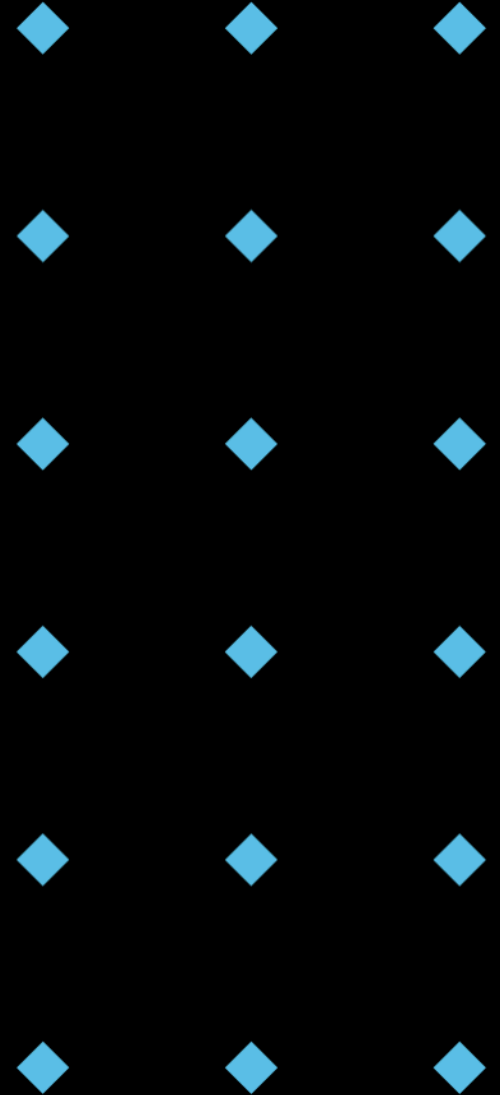
OOP is built upon several fundamental principles. We will cover these in detail in later sessions:

- ◆ **Encapsulation:** Bundling data (attributes) and methods that operate on the data within a single unit (object), and restricting direct access to some components.
- ◆ **Abstraction:** Hiding complex implementation details and exposing only essential features of an object.
- ◆ **Inheritance:** Creating new classes (subclasses) that reuse, extend, and modify the behaviour defined in existing classes (superclasses).
- ◆ **Polymorphism:** Allowing objects of different classes to respond to the same message (method call) in different, class-specific ways.
- ◆ Today: Just know the names and the very basic idea.



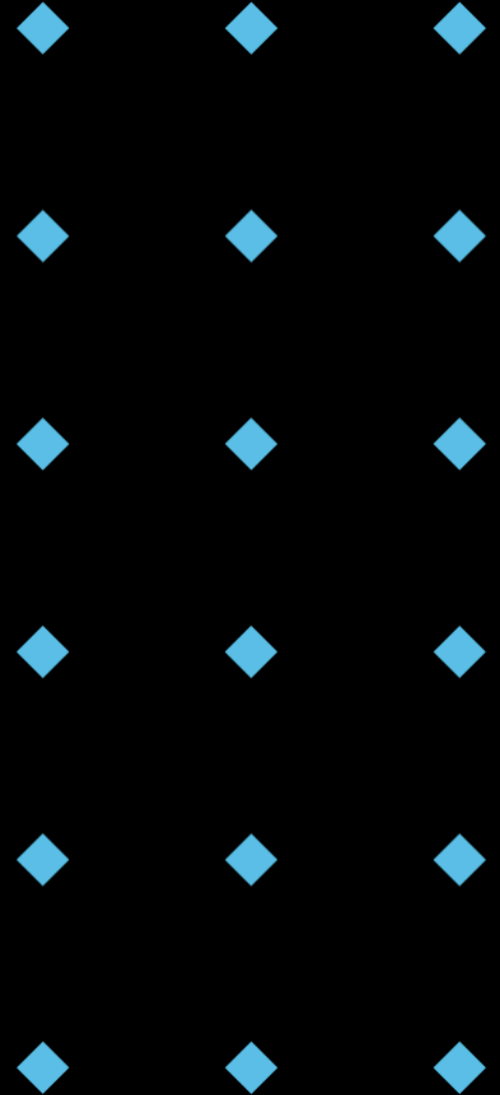
Benefit 1: Modularity & Organization

- ◆ Concept: OOP naturally breaks down a complex system into **smaller, self-contained objects**.
- ◆ Result: Each object represents a distinct part of the system with its own data and responsibilities. This makes the overall system structure clearer and easier to understand.
- ◆ Analogy: Organizing a large project into different specialized teams (objects), each responsible for their specific tasks.



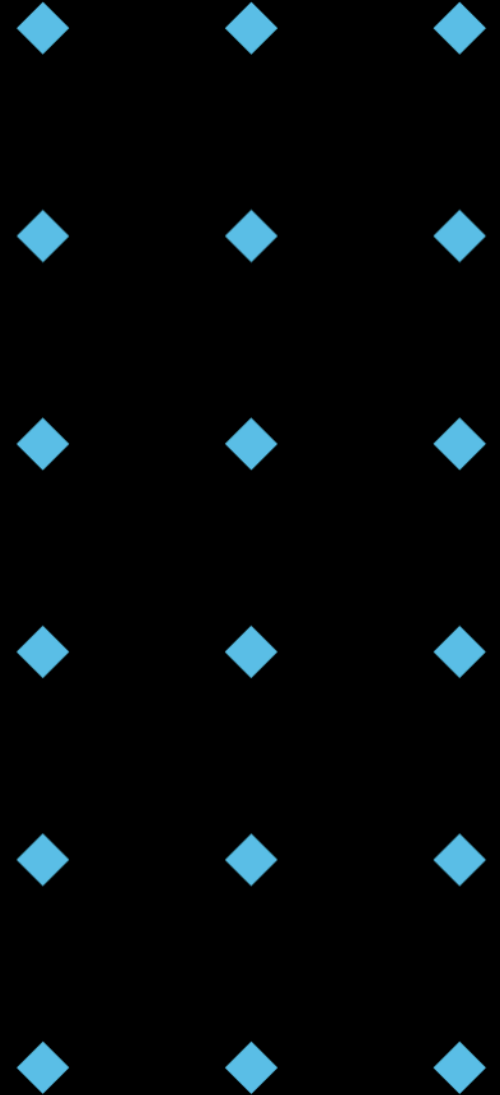
Benefit 2: Reusability

- ◆ **Concept:** Objects and classes defined once can be reused in different parts of the program or even in entirely different programs. Inheritance (which we'll cover later) is a key enabler of reusability.
- ◆ **Result:** Reduces redundant code, saves development time, and improves consistency.
- ◆ **Analogy:** Using standard, pre-fabricated components (like windows or doors) when building multiple houses.



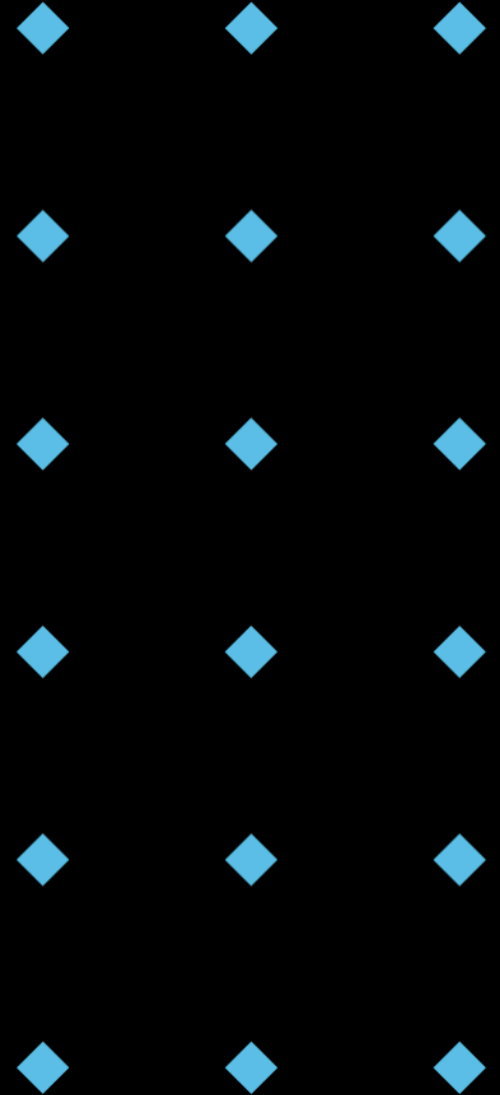
Benefit 3: Maintainability & Scalability

- ◆ **Concept:** Changes within one object are less likely to break other parts of the system (due to encapsulation). New features can often be added by creating new objects or classes.
- ◆ **Result:** Easier to fix bugs, update features, and add new functionality without disrupting the entire system. Systems can grow more easily.
- ◆ **Analogy:** If one component in your modular stereo system breaks (e.g., the CD player), you can replace or repair that component without affecting the speakers or amplifier.



Benefit 4: Collaboration

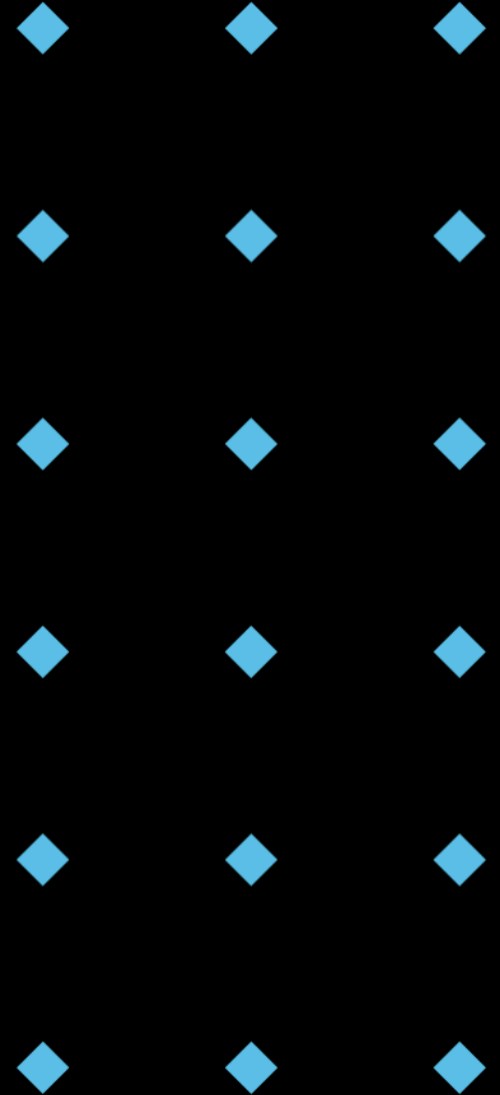
- ◆ **Concept:** Different developers can work on different classes or objects simultaneously with less interference.
- ◆ **Result:** Facilitates teamwork on large projects. Clear interfaces between objects define how they interact.
- ◆ **Analogy:** Different construction crews working on different parts of a building (plumbing, electrical, framing) based on a shared blueprint and defined connection points.



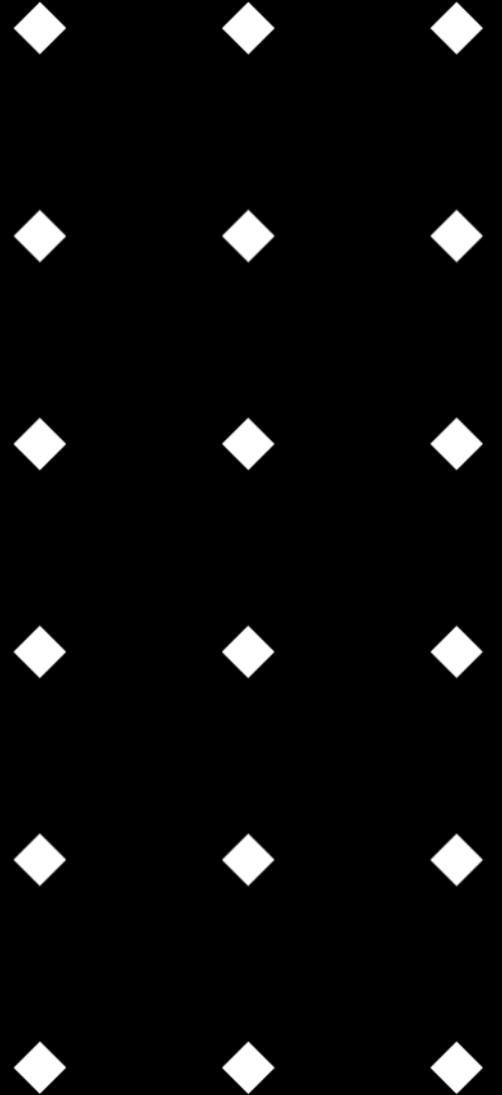
OOP in the Real World

Where is OOP used? Extensively!

- ◆ **Graphical User Interfaces (GUIs):** Buttons, windows, menus are often objects.
- ◆ **Game Development:** Characters, enemies, items, environments are objects.
- ◆ **Web Frameworks:** (e.g., Django, Flask in Python) use objects to represent requests, responses, database models.
- ◆ **Operating Systems:** Files, directories, processes can be modelled as objects.
- ◆ **Data Science & AI Libraries:** (e.g., Scikit-learn, TensorFlow/PyTorch) use classes extensively to represent models, datasets, transformations.



Class and Object



Analogy: Blueprint vs. House

- ◆ **Class:** A **blueprint** or **template** for creating objects.
 - ◆ Defines the **properties (attributes)** and **behaviours (methods)** that all objects of that type will share.
 - ◆ Example: The architectural blueprint for a specific house design.
- ◆ **Object:** An instance of a class. A **concrete realization** of the blueprint.
 - ◆ Has its own specific **state** (values for its **attributes**).
 - ◆ Can perform the **actions** defined by the class.
 - ◆ Example: An actual house built according to the blueprint. You can build multiple houses (**objects**) from the same blueprint (**class**).

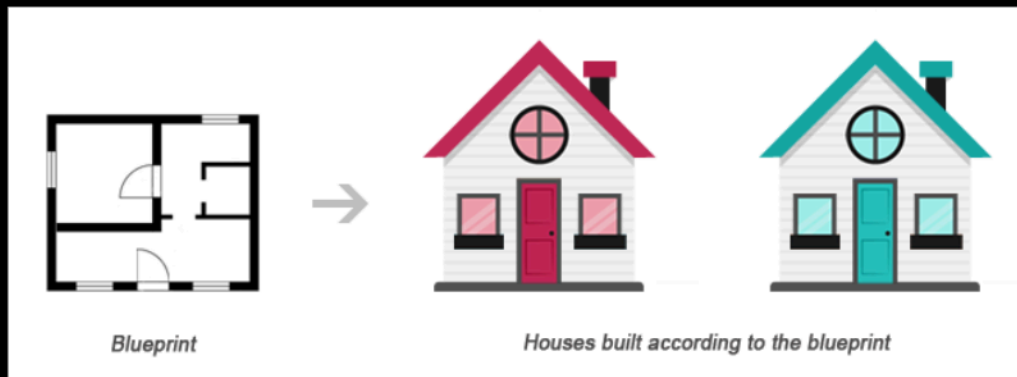


Image from Pradeep K, October 21, 2019, <https://www.devopsschool.com/blog/complete-reference-guide-of-php-classes-objects/>

Defining a Class in Python

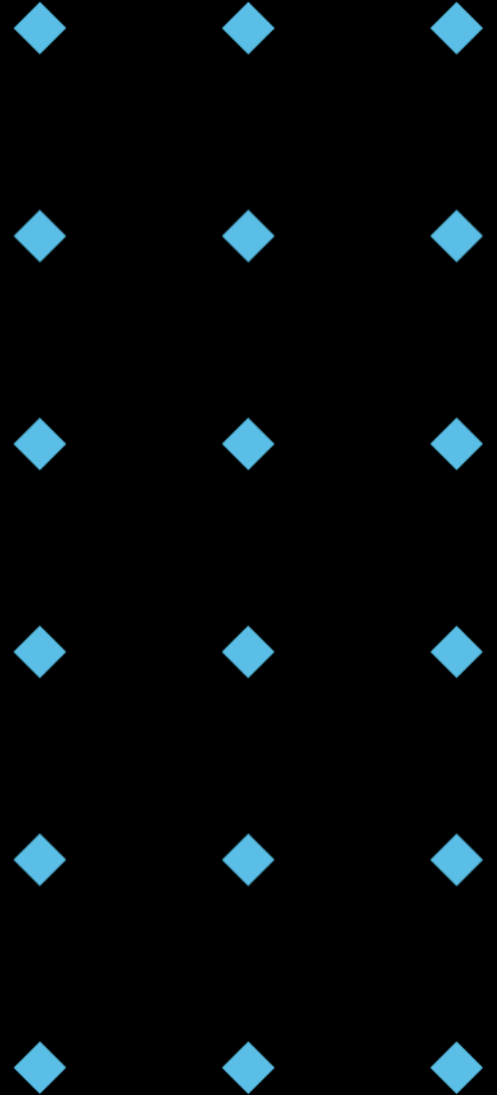
- ♦ **Syntax:** Use the `class` keyword, followed by the class name (use *CamelCase* convention), and a colon `:`. The indented block contains the class definition.
- ♦ `pass` Keyword: A **placeholder** indicating an empty block. Useful when starting a definition.
- ♦ Example:

```
1  class Dog: # Class names use CamelCase
2      pass  # Indicates an empty class definition for now
3
4  class Cat:
5      pass
```

Creating Objects (Instantiation)

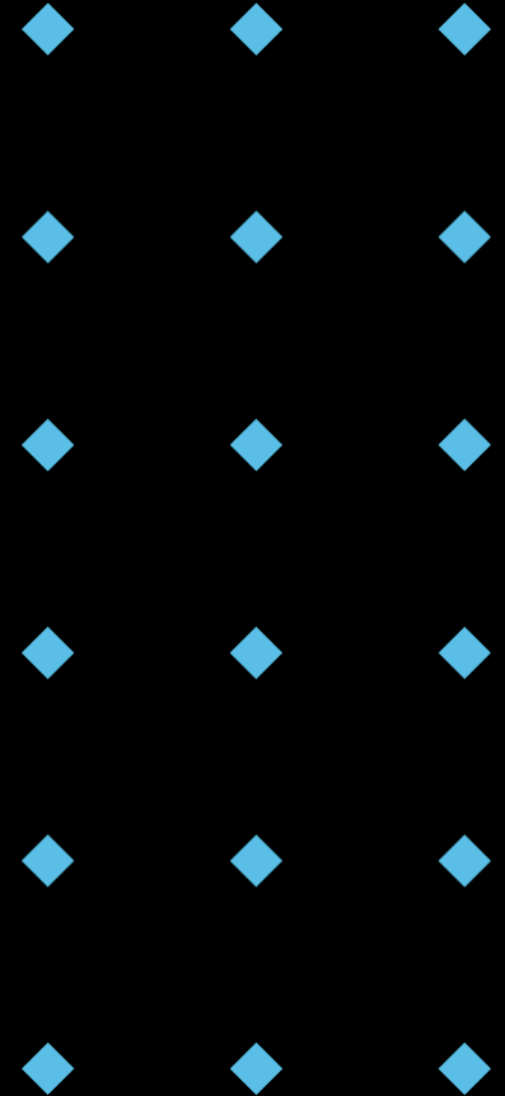
- ◆ **Concept:** Creating an **actual instance (object)** from a class blueprint.
- ◆ **Syntax:** Call the **class name** as if it were a function, using parentheses `()`.
- ◆ **Example:**

```
1  class Dog:
2      pass
3
4  # Create two Dog objects (instances)
5  my_dog = Dog()
6  another_dog = Dog()
7
8  print(my_dog)
9  print(another_dog)
10 print(type(my_dog))
11
12 # Output:
13 # <__main__.Dog object at 0x103009a90>
14 # <__main__.Dog object at 0x102fbf390>
15 # <class '__main__.Dog'>
```



Attributes: Object Data

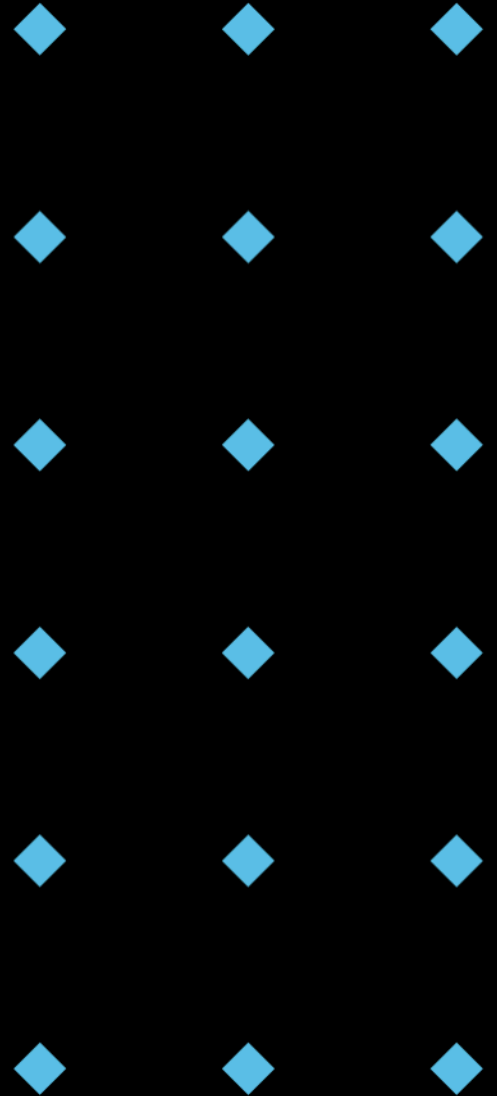
- ◆ **Concept:** Variables that belong to an object, representing its state or characteristics.
- ◆ **Types:**
 - ◆ **Instance Attributes:** Specific to each object (instance). Most common type. (e.g., this specific dog's name, this specific dog's breed).
 - ◆ **Class Attributes:** Shared by all objects of the class. (e.g., all dogs belong to the species "Canis lupus familiaris"). We'll focus on instance attributes first.
- ◆ **Defining Instance Attributes:** Typically done within a special method called the constructor.



The Constructor: `__init__` Method

- ♦ **Purpose:** A special method automatically called when you create a new object (instantiate a class). Used to initialize the object's attributes.
- ♦ **Name:** `__init__` (double underscores before and after "init"). These are often called "dunder" methods or magic methods.
- ♦ **`self` Parameter:** The first parameter of `__init__` (and most instance methods) is always `self`. It represents the object instance being created. Python passes this automatically.
- ♦ **Syntax:**

```
1  class Dog:
2      # The constructor method
3      def __init__(self, name, breed):
4          # 'self' refers to the instance being created
5          # Assigning values passed during creation to instance attributes
6          self.name = name
7          self.breed = breed
8          print(f"Dog object '{self.name}' created!")
```



Creating Objects with `__init__`

- ◆ **Process:** When you call the class name with arguments `Dog("Buddy", "Golden Retriever")`, Python does the following:
 - ◆ Creates a new, empty `Dog` object in memory.
 - ◆ Calls the `__init__` method on this new object.
 - ◆ Passes the new object as the `self` argument.
 - ◆ Passes the other arguments ("Buddy", "Golden Retriever") to the remaining parameters (`name`, `breed`).
 - ◆ The `__init__` method runs, setting `self.name = "Buddy"` and `self.breed = "Golden Retriever"`.
 - ◆ The newly initialized object is returned and assigned to your variable.

- ◆ **Example:**

```
1  class Dog:
2      # The constructor method
3      def __init__(self, name, breed):
4          # 'self' refers to the instance being created
5          # Assigning values passed during creation to instance attributes
6          self.name = name
7          self.breed = breed
8          print(f"Dog object '{self.name}' created!")
9
10 # Now we MUST provide arguments corresponding to __init__ parameters (after self)
11 dog1 = Dog("Buddy", "Golden Retriever")
12 dog2 = Dog("Lucy", "Poodle")
```



Accessing Attributes

- ◆ **Concept:** Reading the values stored in an object's attributes.
- ◆ **Syntax:** Use the dot (.) notation: `object_variable.attribute_name`.
- ◆ **Example:**

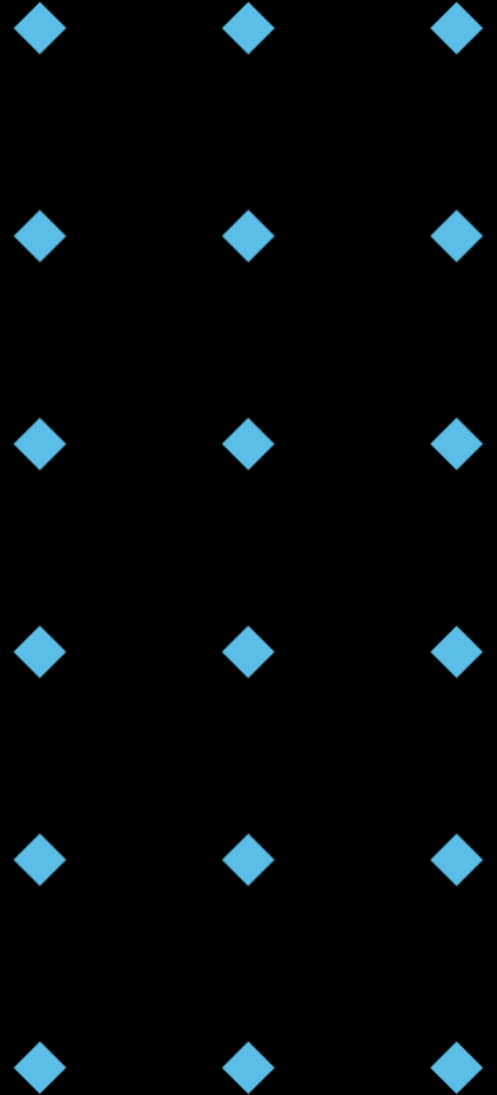
```
1  class Dog:
2      # The constructor method
3      def __init__(self, name, breed):
4          # 'self' refers to the instance being created
5          # Assigning values passed during creation to instance attributes
6          self.name = name
7          self.breed = breed
8          print(f"Dog object '{self.name}' created!")
9
10 # Now we MUST provide arguments corresponding to __init__ parameters (after self)
11 dog1 = Dog("Buddy", "Golden Retriever") # Dog object 'Buddy' created!
12 dog2 = Dog("Lucy", "Poodle") # Dog object 'Lucy' created!
13
14 print(f"{dog1.name} is a {dog1.breed}.") # Buddy is a Golden Retriever.
15 print(f"{dog2.name} is a {dog2.breed}.") # Lucy is a Poodle.
16
17 # Attributes can often be modified directly
18 dog2.breed = "Standard Poodle"
19 print(f"{dog2.name} is now a {dog2.breed}.") # Lucy is now a Standard Poodle.
```

Interactive Question

```
1  class Car:
2      def __init__(self, make, model, year):
3          self.make = make
4          self.model = model
5          self.year = year
6          self.odometer_reading = 0 # Default value
7
8  my_car = Car("Toyota", "Camry", 2021)
9  your_car = Car("Honda", "Civic", 2022)
10 your_car.year = 2023
```

◆ Question: What will the following lines print?

- ◆ `print(my_car.model)`
- ◆ `print(your_car.year)`
- ◆ `print(my_car.odometer_reading)`



Methods: Object Behaviour

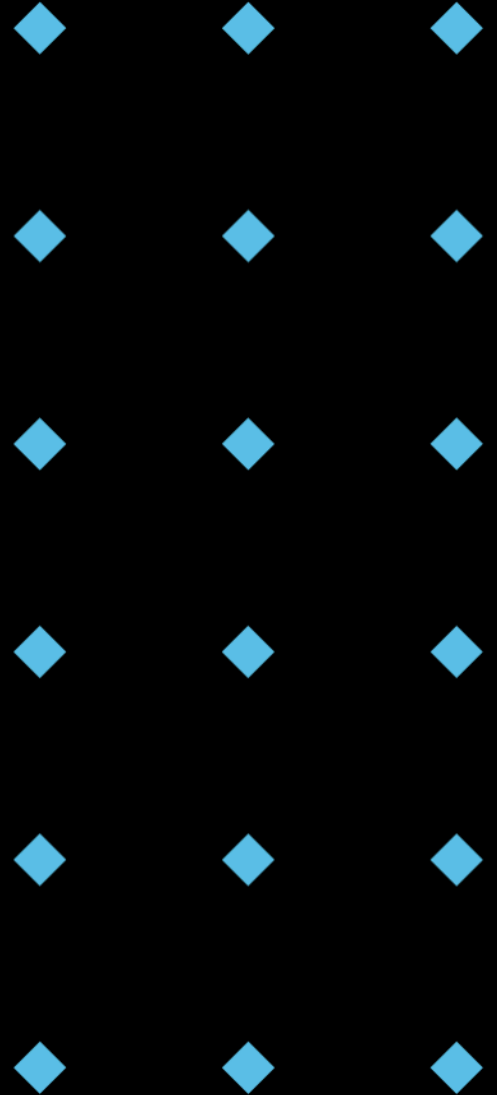
- ◆ **Concept:** Functions defined inside a class that **operate** on the object's data (attributes). Define what an object can do.
- ◆ **Syntax:** Define using `def` inside the class block. Like `__init__`, instance methods take `self` as their first parameter.
- ◆ Example:

```
1  class Dog:
2      def __init__(self, name, breed):
3          self.name = name
4          self.breed = breed
5
6      # This is an instance method
7      def bark(self): # Takes self to access instance data
8          print(f"{self.name} says Woof!")
9
10     def describe(self):
11         print(f"This dog is {self.name}, a {self.breed}.")
```

Calling Methods

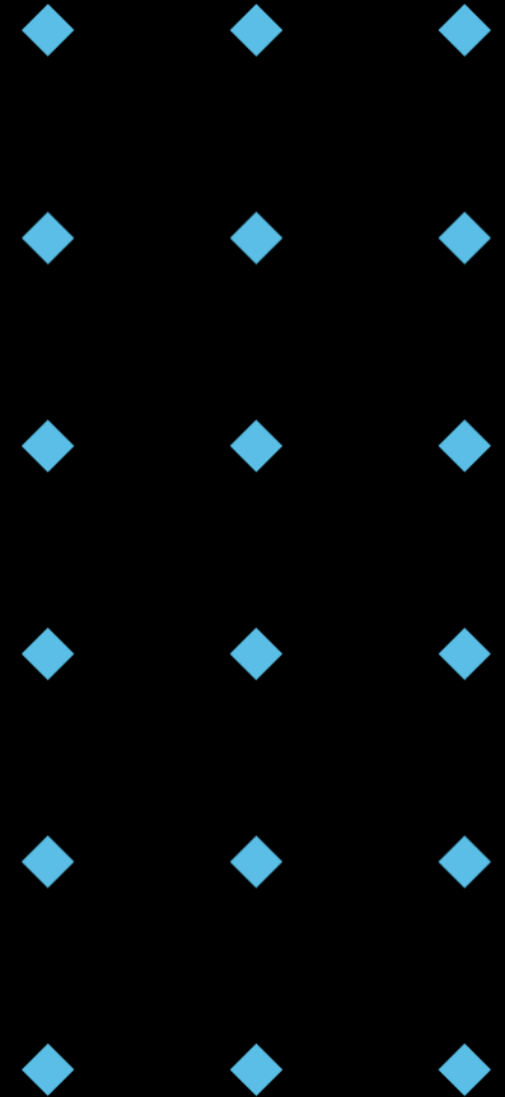
- ◆ **Concept:** Executing an action defined by a method on a specific object.
- ◆ **Syntax:** Use the dot (.) notation: `object_variable.method_name(arguments)`. Python automatically passes the object (`object_variable`) as the `self` argument.
- ◆ **Example:**

```
1 class Dog:
2     def __init__(self, name, breed):
3         self.name = name
4         self.breed = breed
5
6     # This is an instance method
7     def bark(self): # Takes self to access instance data
8         print(f"{self.name} says Woof!")
9
10    def describe(self):
11        print(f"This dog is {self.name}, a {self.breed}.")
12
13    dog1 = Dog("Buddy", "Golden Retriever")
14    dog2 = Dog("Lucy", "Poodle")
15
16    # Call methods on the objects
17    dog1.bark()      # Buddy says Woof!
18    dog2.bark()      # Lucy says Woof!
19    dog1.describe()  # This dog is Buddy, a Golden Retriever.
20    dog2.describe()  # This dog is Lucy, a Poodle.
```



Putting It Together: Car Example

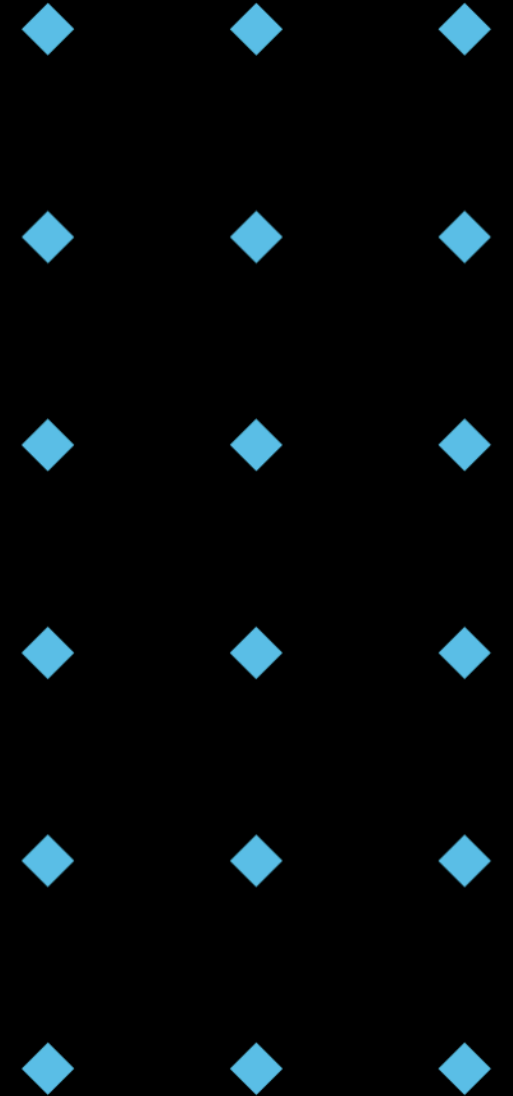
```
1  class Car:
2      def __init__(self, make, model, year):
3          self.make = make
4          self.model = model
5          self.year = year
6          self.odometer_reading = 0 # Instance attribute
7
8      def get_descriptive_name(self): # Method
9          """Return a neatly formatted descriptive name."""
10         long_name = f"{self.year} {self.make} {self.model}"
11         return long_name.title()
12
13     def read_odometer(self): # Method
14         """Print the car's mileage."""
15         print(f"This car has {self.odometer_reading} kms on it.")
16
17     def drive(self, kms): # Method with parameter
18         """Simulate driving and update odometer."""
19         if kms > 0:
20             self.odometer_reading += kms # Modify attribute
21             print(f"Drove {kms} kms.")
22         else:
23             print("Cannot drive negative kms!")
24
25 # Using the class
26 my_car = Car("audi", "a4", 2025)
27 print(my_car.get_descriptive_name()) # 2025 Audi A4
28 my_car.read_odometer()              # This car has 0 kms on it.
29 my_car.drive(100)                   # Drove 100 kms.
30 my_car.read_odometer()              # This car has 100 kms on it.
```



Interactive Exercise

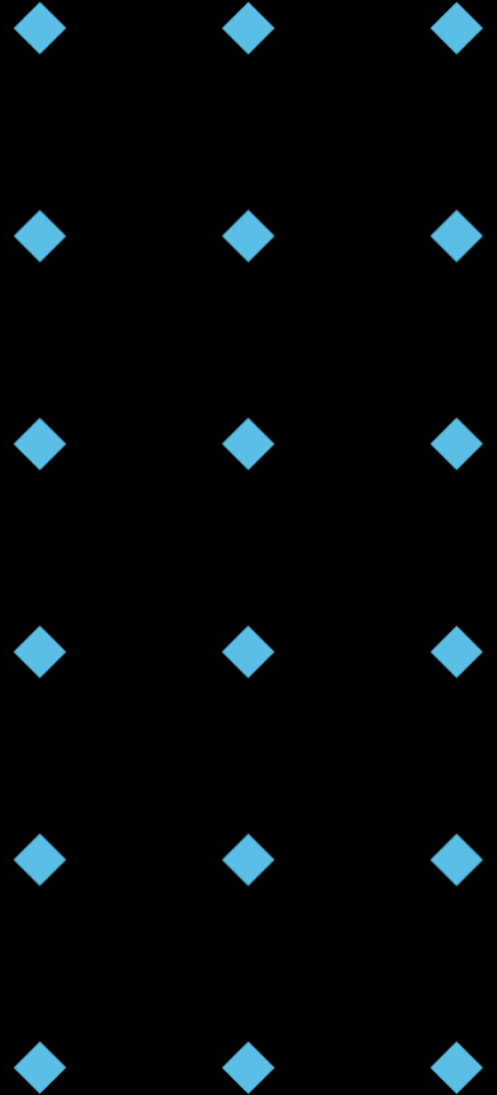
- ◆ Task: Define a simple *Student* class.
- ◆ It should have an `__init__` method that takes *name* and *student_id* as arguments and stores them as attributes.
- ◆ It should also have a method called *display_info* that prints the student's name and ID in a readable format (e.g., "Student Name: Alice, ID: 12345").
- ◆ Code without AI Copilot! We will cover AI Copilot in later sessions.

```
1  class Student:
2      # Your __init__ method here
3      def __init__(self, name, student_id):
4          # Initialize attributes
5          pass # Replace pass
6
7      # Your display_info method here
8      def display_info(self):
9          # Print the info
10         pass # Replace pass
11
12     # Create a student object and test your method
13     # student1 = Student(...)
14     # student1.display_info()
```



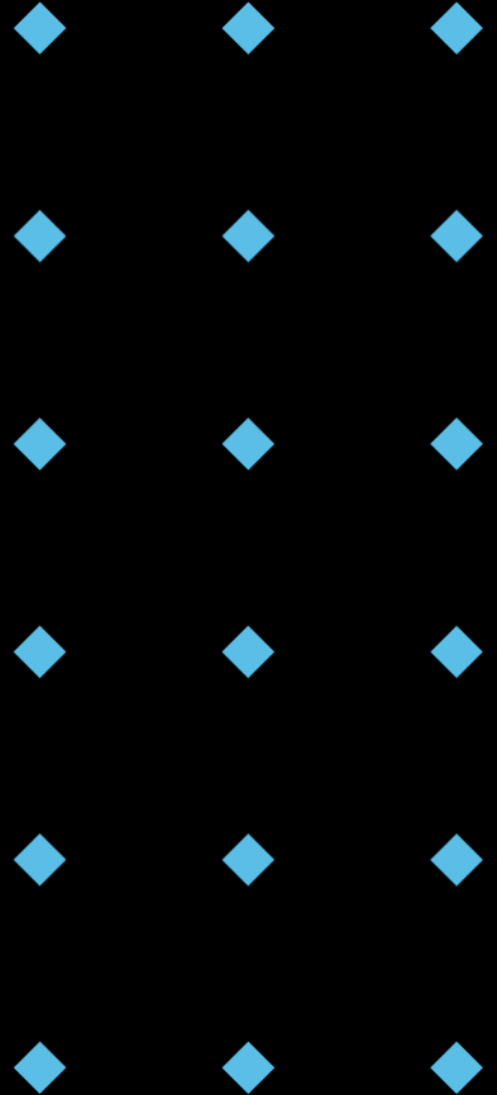
Interactive Exercise - Solution

```
1  class Student:
2      def __init__(self, name, student_id):
3          """Initializes student attributes."""
4          self.name = name
5          self.student_id = student_id
6          print(f"Student object created for {self.name}")
7
8      def display_info(self):
9          """Displays the student's information."""
10         print(f"Student Name: {self.name}, ID: {self.student_id}")
11
12     # Create a student object and test your method
13     student1 = Student("Alice", 12345) # Student object created for Alice
14     student2 = Student("Bob", 67890)   # Student object created for Bob
15
16     student1.display_info() # Student Name: Alice, ID: 12345
17     student2.display_info() # Student Name: Bob, ID: 67890
```

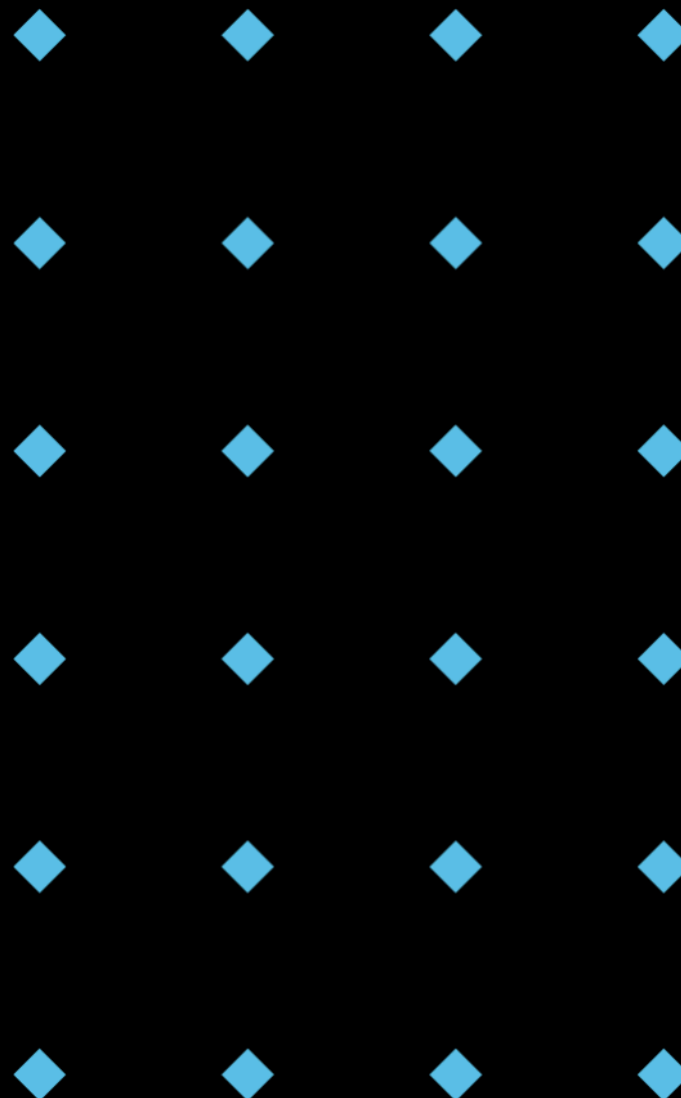


Key Takeaways

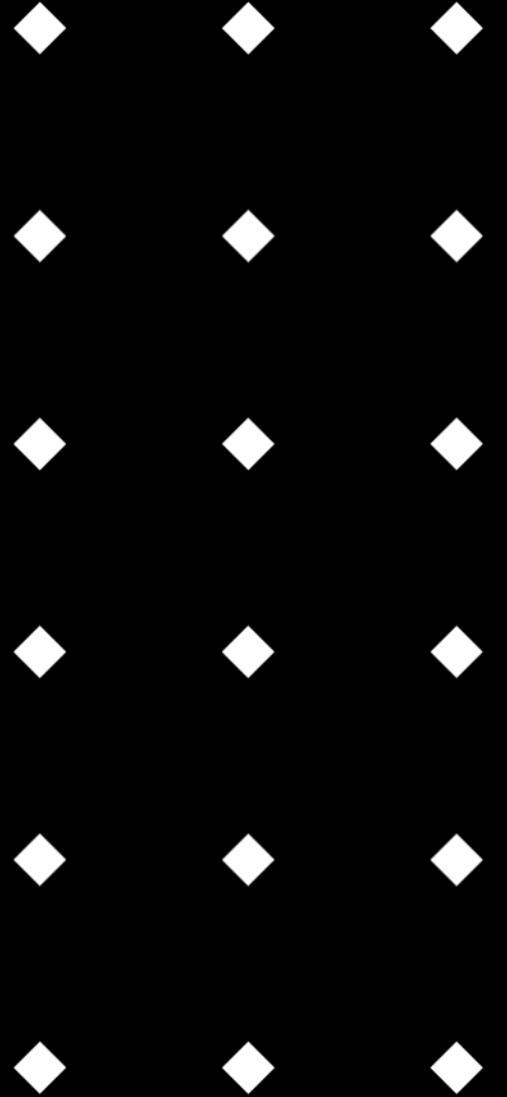
- ♦ **Python Basics:** Reviewed essential syntax, types, control flow, functions, modules.
- ♦ OOP Introduction:
 - ♦ Paradigm focusing on objects (**data + behaviour**).
 - ♦ Benefits: **Modularity, Reusability, Maintainability, Collaboration**.
- ♦ Class & Object Fundamentals:
 - ♦ **Class:** Blueprint (*class ClassName:*).
 - ♦ **Object:** Instance created from class (*my_obj = ClassName()*).
 - ♦ **`__init__` (Constructor):** Initializes object state (attributes), takes *self*.
 - ♦ **Attributes:** Object data (*self.attribute = value*). Accessed via *object.attribute*.
 - ♦ **Methods:** Object behavior (*def method_name(self, ...):*). Called via *object.method()*.
 - ♦ ***self*:** Reference to the instance itself, passed automatically.



Q & A



Lab



Start with Python

- ◆ Step1: Download and install Python 3 Interpreter :

<https://www.python.org/>

- ◆ Step 2: Install an Integrated Development Environment (IDE):

<https://code.visualstudio.com/>

(We recommend Visual Studio Code, but you may choose your favourite one)

- ◆ Step 3: Use VS Code to create and execute your first Python program by following the tutorial below:

<https://code.visualstudio.com/docs/python/python-tutorial>



<https://www.python.org/downloads/>



<https://code.visualstudio.com/download>

